

1.1 KOD Program tworzy poprawny pusty plik SVG.

Pyta użytkownika o nazwę pliku i go tworzy, następnie pyta czy ma się ten pusty plik otworzyć. Znajduje się to w pliku svg.cpp, linie 13-57.

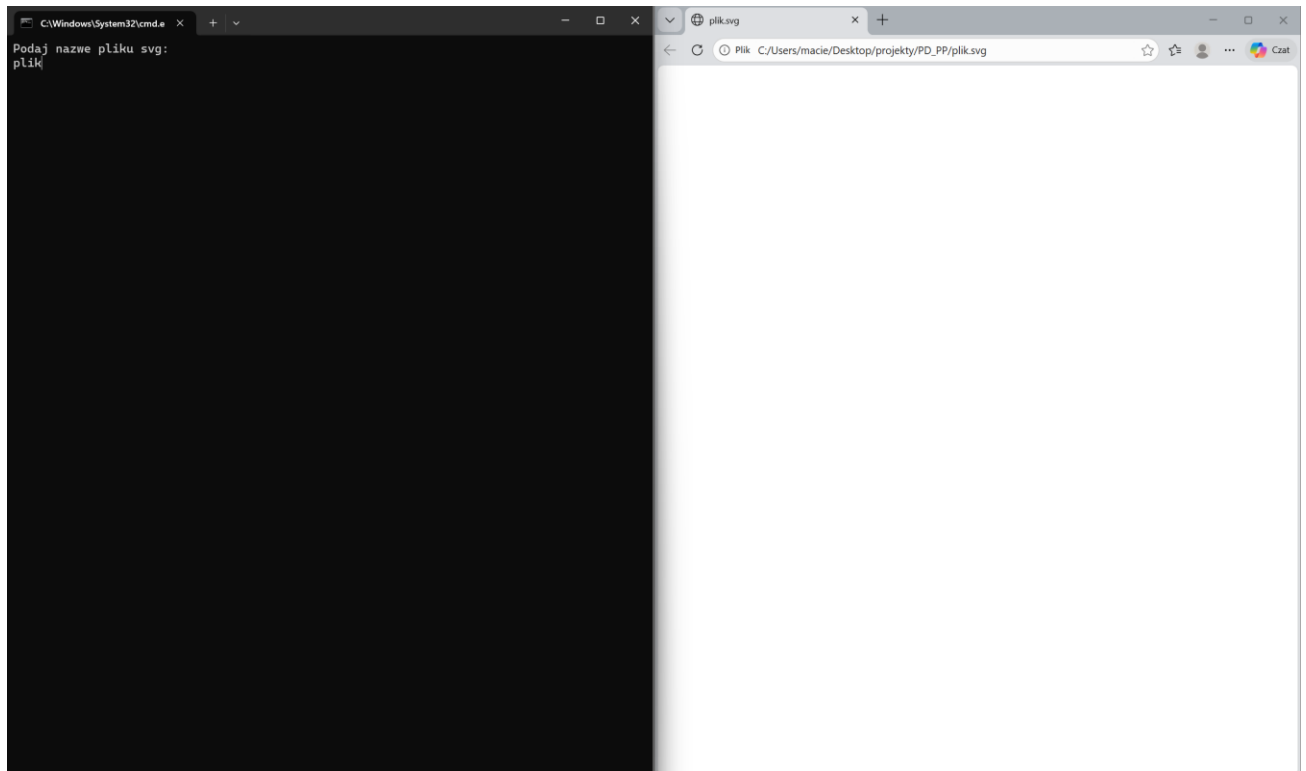
```
string nazwapliku;
cout << "Podaj nazwe pliku svg: " << endl;
cin >> nazwapliku; // pobieranie od uzytkownika nazwy pliku
nazwapliku += ".svg";

ofstream plik(nazwapliku);
if(!plik.is_open())          //sprawdzenie czy plik sie otworzyl
{
    cout << "Błąd otwarcia pliku" << endl;
    return;
}

plik << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
plik << "<svg width=\"500\" height=\"500\"
xmlns=\"http://www.w3.org/2000/svg\">\n";

plik << "</svg>\n";
plik.close();
cout << "Czy chcesz stworzyc swoj pusty plik svg? (tak/nie)" << endl;
while (true)
{
    string zgoda;
    cin >> zgoda;
    if(zgoda == "tak")
    {
        string cmd = "start " + nazwapliku;
        system(cmd.c_str());
        break;
    }
    else if (zgoda == "nie")
    {
        break;
    }
    else
    {
        cout << "Wpisz tak/nie" << endl;
    }
}
system("cls"); }
```

1.1 SCREEN



1.2 KOD Plik zawiera 3 kształty podstawowe (<circle>, <rect>, <line>), o zadanych przez użytkownika współrzędnych , kolorach i rozmiarze.

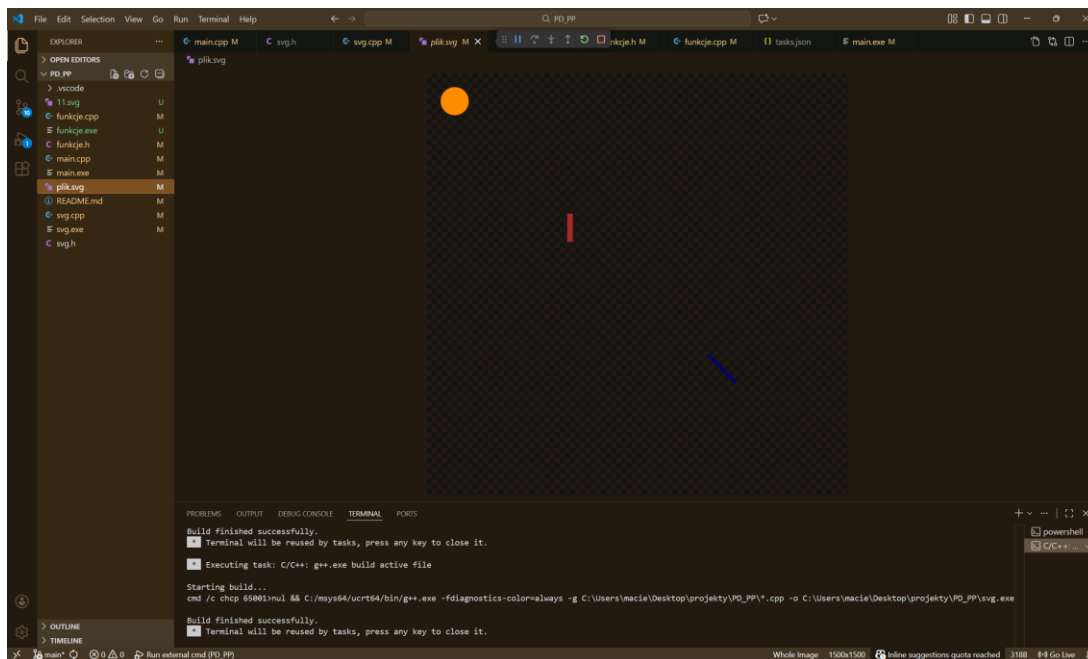
Fragment kodu, gdzie wczytywane jest koło, został użyta funkcja “wczytaj_int” która wypłauje, czy użytkownik wpisał liczbę. Obliczana jest też tutaj granica koła, która jest później wykorzystana do wykrywania z nią kolizji innych figur, w dalszej części to samo dla prostokąta i linii. Ten fragment kodu znajduje się w pliku svg.cpp, linie 84-103.

```
//KOŁO
    int cx, cy, r, nrkoloru;
string ckolor;
cx = wczytaj_int("Podaj wspolrzedna x srodka kola: "); // pozwalanie na
    wczystanie jedynie liczby
cy = wczytaj_int("Podaj wspolrzedna y srodka kola: ");
r = wczytaj_int("Podaj promien okregu: ");
cout << "Wybierz kolor dla okregu:" << endl;
ckolor = wybor_koloru();

plik << "<circle cx=\"" << cx;
plik << "\" cy=\"" << cy;
plik << "\" r=\"" << r;
plik << "\" fill=\"" << ckolor;
plik << "\" />\n";

//granice kola
int clg = cx - r; // lewa granica kola
int cpg = cx + r; // prawa granica kola
int cgg = cy - r; // gorna granica kola
int cdg = cy + r; // dolna granica kola
```

1.2 SCREEN



```
C:\Windows\System32\cmd.exe
Podaj wspolrzeczna y srodka kola: 100
Podaj promien okregu: 50
Wybierz kolor dla okregu:
(1) czerwony
(2) niebieski
(3) zielony
(4) czarny
(5) secret
5
Podaj wspolrzeczna x lewego gornego rogu prostokata: 100
Podaj wspolrzeczna y lewego gornego rogu prostokata: 100
Podaj szerokosc prostokata: 20
Podaj wysokosc prostokata: 20
Prostokat nachodzi na okrag, podaj inne dane!!!

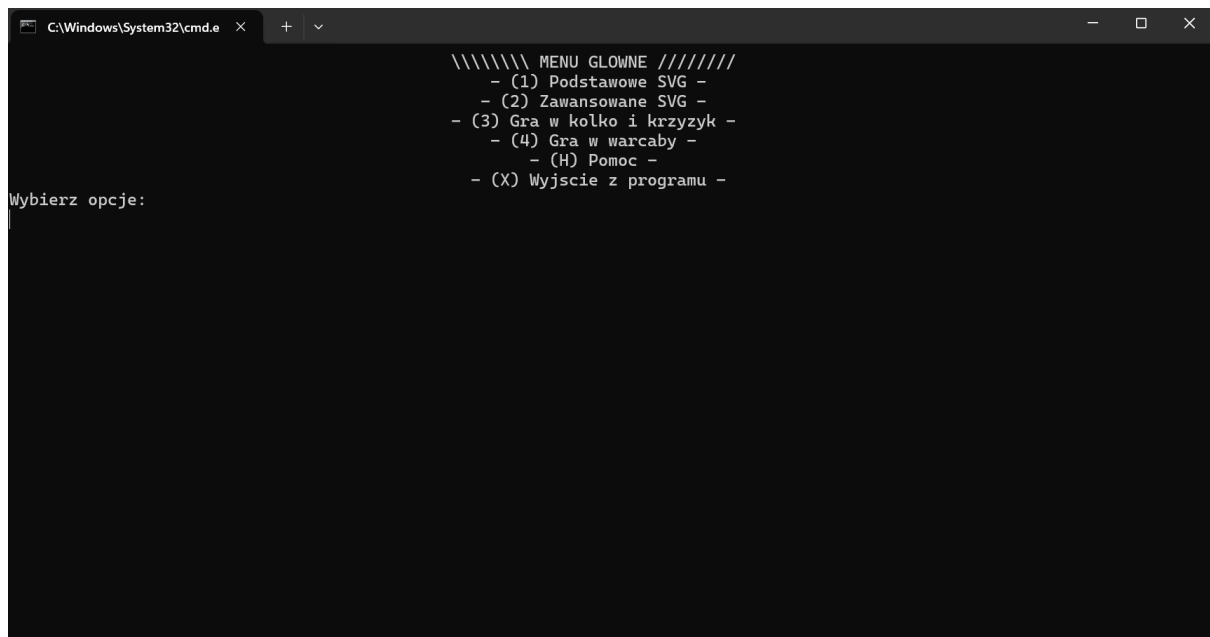
Podaj wspolrzeczna x lewego gornego rogu prostokata: 500
Podaj wspolrzeczna y lewego gornego rogu prostokata: 500
Podaj szerokosc prostokata: 20
Podaj wysokosc prostokata: 100
Wybierz kolor prostokata:
(1) czerwony
(2) niebieski
(3) zielony
(4) czarny
(5) secret
5
Podaj wspolrzeczna x pierwszego konca linii: 1000
Podaj wspolrzeczna y pierwszego konca linii: 1000
Podaj wspolrzeczna x drugiego konca linii: 1100
Podaj wspolrzeczna y drugiego konca linii: 1100
Podaj szerokosc linii: 10
Wybierz kolor linii:
(1) czerwony
(2) niebieski
(3) zielony
(4) czarny
(5) secret
5
```

1.3 KOD Opracowanie menu programu.

Przedstawiony kod wypisuje opcje menu oraz posiada switch ktory te funkcje z menu wywołuje. Skorzystałem z getline zeby wyeliminować problem w którym program nie działał poprawnie, gdy wpisało się więcej niż jedna cyfra lub litera. Jest użyta funkcja wys() która wyśrodkowuje tekst na podstawie szerokości konsoli. A cin >> ws w getline usuwa białe znaki. Fragment kodu znajduje się w pliku main.cpp, linie 26-51

```
int main() {  
  
    string opcje;  
    srand(time(nullptr));  
    system("cls");  
    // Menu glowne  
    do  
    {  
        wys( "\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\ MENU GLOWNE \\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\\");  
        wys("- (1) Podstawowe SVG -");  
        wys("- (2) Zawansowane SVG -");  
        wys("- (3) Gra w kolko i krzyzyk -");  
        wys("- (4) Gra w warcaby - ");  
        wys("- (H) Pomoc -");  
        wys("- (X) Wyjście z programu -");  
        cout << "Wybierz opcje: " << endl;  
        getline(cin >> ws, opcje);  
        if(opcje.size() == 1 && (opcje[0] == '1' || opcje[0] == '2' ||  
opcje[0] == '3' || opcje[0] == '4' || opcje[0] == 'H' ||  
opcje[0] == 'h' || opcje[0] == 'X' || opcje[0] == 'x'))  
        {  
            switch (opcje[0])  
            {  
                system("cls");  
                svg_pod();  
                system("cls");  
                break;  
            }  
        }  
    }  
}
```

1.3 SCREEN



A screenshot of a Windows command prompt window. The title bar shows the path 'C:\Windows\System32\cmd.e' and standard window controls. The command prompt displays a menu titled 'MENU GLOWNE' with several options. The prompt 'Wybierz opcje:' is visible on the left side of the screen.

```
\\ \\ \\ \\ MENU GLOWNE // // // //
- (1) Podstawowe SVG -
- (2) Zawansowane SVG -
- (3) Gra w kolko i krzyzyk -
- (4) Gra w warcaby -
- (H) Pomoc -
- (X) Wyjście z programu -

Wybierz opcje:
```

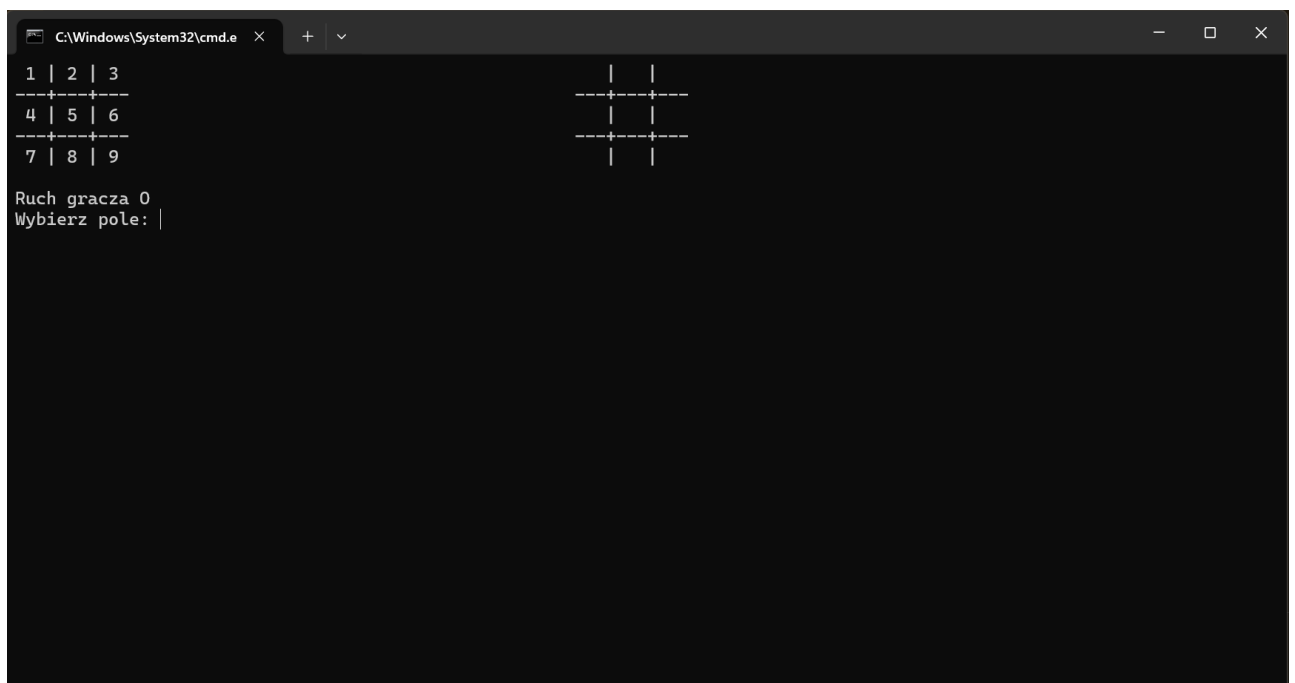
2.0 KOD Wyświetlanie planszy do kółka i krzyżyka w konsoli z ASCII.

Wyświetla plansze gry do kółka i krzyżyk oraz plansze z numerami komórek, które są wolne.
Fragment kodu znajduje się w pliku XO.cpp, linie 242-270

```
void XO_plansza(char XO[3][3], char graf_plansza_XO[3][3])
{

    int w = (szer_cmd() - 30) / 2;
    cout << " " << graf_plansza_XO[0][0] << " | " << graf_plansza_XO[0][1]
        << " | " << graf_plansza_XO[0][2] << " " << setw(w) << XO[0][0]
        << " | " << XO[0][1] << " | " << XO[0][2] << " " << endl;
    cout << "---+---+---" << setw(w+9) << "---+---+---" << endl;
    cout << " " << graf_plansza_XO[1][0] << " | " << graf_plansza_XO[1][1]
        << " | " << graf_plansza_XO[1][2] << " " << setw(w) << XO[1][0]
        << " | " << XO[1][1] << " | " << XO[1][2] << " " << endl;
    cout << "---+---+---" << setw(w+9) << "---+---+---" << endl;
    cout << " " << graf_plansza_XO[2][0] << " | " << graf_plansza_XO[2][1]
        << " | " << graf_plansza_XO[2][2] << " " << setw(w) << XO[2][0]
        << " | " << XO[2][1] << " | " << XO[2][2] << " " << endl;
```

2.0 SCREEN



```
C:\Windows\System32\cmd.e  x + v
1 | 2 | 3
---+---+---
4 | 5 | 6
---+---+---
7 | 8 | 9

      | |
      + +
      | |
      + +
      | |

Ruch gracza 0
Wybierz pole: |
```

2.1 KOD Działający mechanizmy gry w kółko i krzyżyk (w konsoli) gracz vs gracz – poprawne stawianie znaków i warunek wygranej.

Kod 9 razy (maksymalna ilość tur) pyta gracza, gdzie postawić następny znak.

Sprawdza, czy gracz wpisał odpowiednią komendę i na podstawie tego która jest aktualnie runda stawia odpowiedni znak. Fragmenty znajdują się w XO.cpp.

Linie 288-481. Warunek wygrany sprawdza czy w jednej linii pojawia się X lub O i na podstawie rundy sprawdza kto wygrał.

Linie 288-481. Stawianie znaku:

```
if(n >= 1 && n <= 9) //Stawianie znaków na planszy
{
    int wiersz = (n - 1) / 3;
    int kolumna = (n - 1) % 3;
    if(XO[wiersz][kolumna] != 'X' && XO[wiersz][kolumna] != 'O')
    {

        if(i % 2 == 0)
        {
            XO[wiersz][kolumna] = 'O';
            graf_plansza_XO[wiersz][kolumna] = ' ';

        }
        else
        {
            XO[wiersz][kolumna] = 'X';
            graf_plansza_XO[wiersz][kolumna] = ' ';

        }
        system("cls");
        XO_svg_plansza(XO, XO_graf, XO_ods);
        XO_plansza(XO, graf_plansza_XO, czasO, czasX, XO_czas);

    }
    else
    {
        cout << "Pole juz zajete. Spróbuj ponownie." << endl;
        i--;
        continue;
    }
}
```


2.1 SCREEN



```
C:\Windows\System32\cmd.e  X + v - □ X

  | 2 | 3
--+---+
4 |  | 6
--+---+
7 | 8 |

Ruch gracza X
Wybierz pole: |

      X |  | 
      ---+---
      | 0 | 
      ---+---
      |  | 0

KONIEC GRY!!!

  O | X | O
  ---+---
  X | O | X
  ---+---
  O |  | 

$$$ GRACZ O WYGRYWA $$$

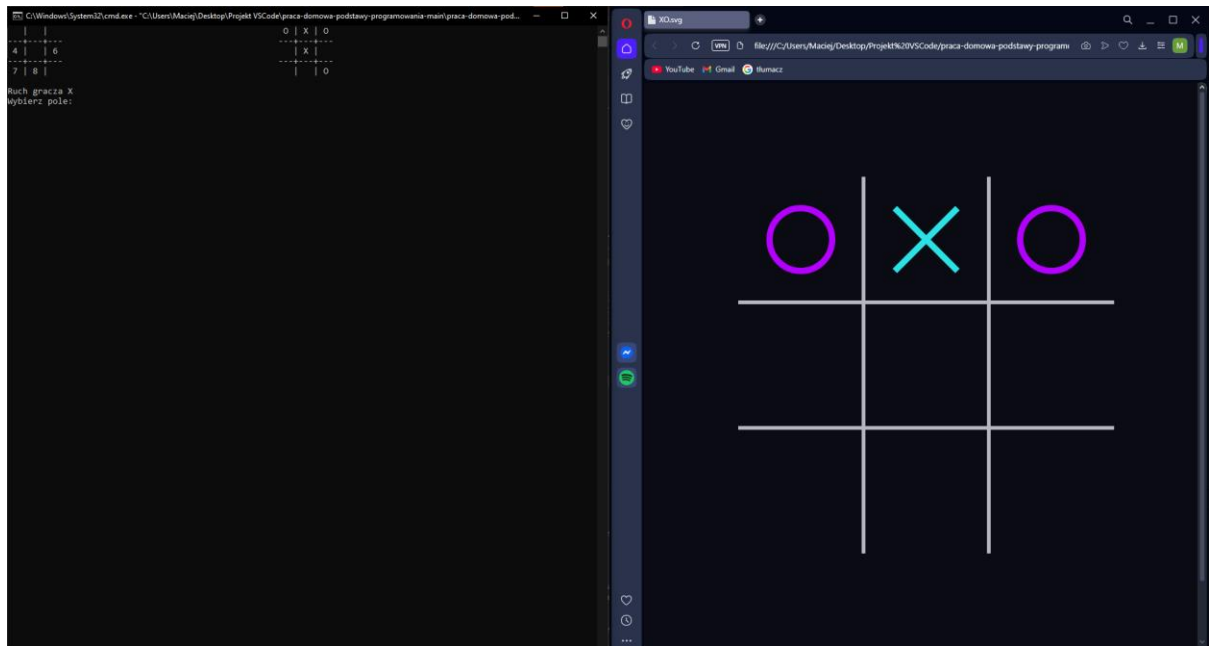
Przywróc okno do oryginalnego rozmiaru.
Nacisnij Enter, aby kontynuować
```

2.2 KOD Generowanie planszy gry, w pliku .SVG i aktualizacja krok po kroku w trakcie gry (można odświeżać plik w przeglądarce, żeby widzieć grę „na żywo”).

Kod rysuje prosta plansze do kółka i krzyżyk, następnie wylicza miejsce postawienia X i O. Figura będzie postawiona jeżeli takowa się pojawi w tablicy gry i nie będzie tam innej figury. Plik XO.cpp, funkcja void XO_svg_plansza(char XO[3][3]), linie 182-239.

```
plik << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
plik << "<svg width=\"1000\" height=\"1000\"
xmlns=\"http://www.w3.org/2000/svg\">\n";
plik << "<rect x=\" 0\" y=\" 0\" width=\" 1500\" height=\"1500\"
fill=\" #0B0B12\" />\n";
plik << "<line x1=\"350\" y1=\"150\" x2=\"350\" y2=\"750\"
stroke=\"#B8B8C2\" stroke-width=\"6\" />\n";
plik << "<line x1=\"550\" y1=\"150\" x2=\"550\" y2=\"750\"
stroke=\"#B8B8C2\" stroke-width=\"6\" />\n";
plik << "<line x1=\"150\" y1=\"350\" x2=\"750\" y2=\"350\"
stroke=\"#B8B8C2\" stroke-width=\"6\" />\n";
plik << "<line x1=\"150\" y1=\"550\" x2=\"750\" y2=\"550\"
stroke=\"#B8B8C2\" stroke-width=\"6\" />\n";
for( int i = 0; i < 3; i++)
{
    for(int j = 0; j < 3; j++)
    {
        if(XO[i][j] == 'X')
        {
            int x1 = 200 + j * 200;
            int y1 = 200 + i * 200;
            int x2 = 300 + j * 200;
            int y2 = 300 + i * 200;
            plik << "<line x1=\"\" << x1 << \"\" y1=\"\" << y1 << \"\"
x2=\"\" << x2 << \"\" y2=\"\" << y2 << \"\" stroke=\"#2DE2E6\"
stroke-width=\"10\" />\n";
            plik << "<line x1=\"\" << x1 << \"\" y1=\"\" << y2 << \"\" x2=\"\" << x2 <<
\"\" y2=\"\" << y1 << \"\" stroke=\"#2DE2E6\" stroke-width=\"10\" />\n";
        }
        else if(XO[i][j] == 'O')
        {
            int cx = 250 + j * 200;
            int cy = 250 + i * 200;
            plik << "<circle cx=\"\" << cx << \"\" cy=\"\" << cy << \"\"
r=\"50\" stroke=\"#B300FF\" stroke-width=\"10\" fill=\"none\" />\n";
        }
    }
}
plik << "</svg>\n";
plik.close(); system("cls");}
```

2.2 SCREEN



2.3 KOD Dodaj możliwość gry z graczem (człowiek) lub z komputerem (PC)

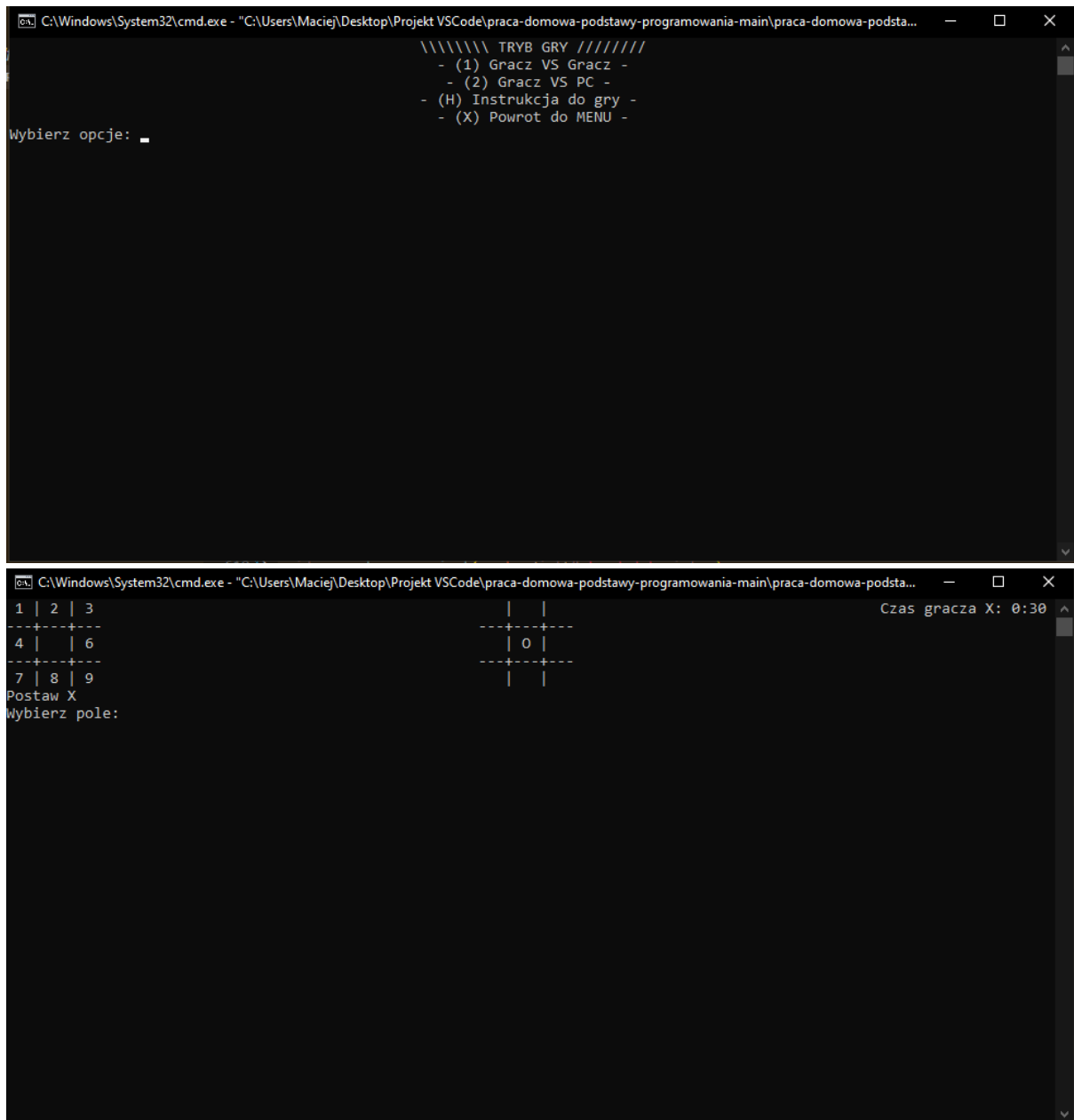
Fragment kodu stawia pierwszy znak komputera. Kod do tego podpunktu znajduje się w pliku XO.cpp w funkcji void gra_XO_PC(), linie 484-896.

```
if(i % 2 == 0 && gracz_start == true || i % 2 != 0 && gracz_start == false)
{
    cout << "Postaw " << gznak << endl;

    n = wczytaj_int("Wybierz pole: ");
}
else
{
    if(kznak == 'O') czasO = 67;
    else if(kznak == 'X') czasX = 67;

    if(i == 0 && gracz_start == false)
    {
        n = 5;
    }
    else if(i == 1 && gracz_start == true)
    {
        if(XO[1][1] == ' ')
        {
            n = 5;
        }
        else
        {
            while(true)
            {
                n = losuj_oddo(1,9);
                int wiersz = (n - 1) / 3;
                int kolumna = (n - 1) % 3;
                if(XO[wiersz][kolumna] != 'X' && XO[wiersz][kolumna] != 'O')
                {
                    break;
                }
            }
        }
    }
}
```

2.3 SCREEN



The image consists of two screenshots of a Windows command prompt window. The title bar of the window reads: "C:\Windows\System32\cmd.exe - "C:\Users\Maciej\Desktop\Projekt VSCode\praca-domowa-podstawy-programowania-main\praca-domowa-podsta...".

The first screenshot shows a menu for a game titled "TRYB GRY". The menu options are:

```
\\ \\ \\ \\ TRYB GRY \\ \\ \\ \\
- (1) Gracz VS Gracz -
- (2) Gracz VS PC -
- (H) Instrukcja do gry -
- (X) Powrot do MENU -
```

Below the menu, the text "Wybierz opcje: " is followed by a cursor.

The second screenshot shows a tic-tac-toe board. The board is represented by a 3x3 grid of numbers 1 through 9, with dashes indicating the board boundaries. The board is currently empty. The text "Postaw X" is displayed below the board, followed by "Wybierz pole:". In the top right corner, the text "Czas gracza X: 0:30" is visible.

2.4 KOD Dodaj licznik czasu gry dla użytkowników, kończ grę po przekroczeniu czasu gry (np. 1 minuta na gracza).

Kod liczy czas stawiania znaku przez gracza i jeśli ten zostanie przekroczony, gracz przegrywa. Znajduje się w pliku XO.cpp, linie 340-380.

```
auto lc_start = std::chrono::steady_clock::now(); // początek liczenia czasu
    n = wczytaj_int("Wybierz pole: "); // wczytanie n
    auto lc_koniec = std::chrono::steady_clock::now(); // koniec liczenia
czasu
    auto ile_czas = std::chrono::duration_cast<std::chrono::seconds>(lc_koniec
- lc_start).count(); //sprawdzenie ile czasu gracz stawiał znak
    if(i % 2 == 0) // odjęcie czasu stawiania od czasu gracza
    {
        czasO -= ile_czas;
    }
    else
    {
        czasX -= ile_czas;
    }
    if(czasO <= 0) // sprawdzenie czy czas się skończył
    {
        system("cls");
        wys("KONIEC GRY!!!");
        cout << endl;
        XO_plansza_wygrana(XO);
        cout << endl;
        wys(":( CZAS GRACZA O SIE SKONCZYL :(");
        wys("$$$ GRACZ X WYGRYWA $$$");
        break;
    }
    else if(czasX <= 0)
    {
        system("cls");
        wys("KONIEC GRY!!!");
        cout << endl;
        XO_plansza_wygrana(XO);
        cout << endl;
        wys(":( CZAS GRACZA X SIE SKONCZYL :(");
        wys("$$$ GRACZ O WYGRYWA $$$");
        break;
    }
}
```

2.4 SCREEN



```
C:\Windows\System32\cmd.exe - "C:\Users\Maciej\Desktop\Projekt VSCode\praca-domowa-podstawy-programowania-main\praca-domowa-podsta...
| 2 | 3
---+---+---
4 | |
---+---+---
7 | 8 | 9

X | |
---+---+---
| 0 | 0
---+---+---
| |

Czas gracza O: 0:23
Czas gracza X: 0:28

Ruch gracza X
Wybierz pole:
```



```
C:\Windows\System32\cmd.exe - "C:\Users\Maciej\Desktop\Projekt VSCode\praca-domowa-podstawy-programowania-main\praca-domowa-podsta...

KONIEC GRY!!!

X | |
---+---+---
| 0 | 0
---+---+---
| |

:( CZAS GRACZA X SIE SKONCZYL :(
$$$ GRACZ O WYGRYWA $$$

Nacisnij Enter, aby kontynuowac
```

3.1 Gra w warcaby z podstawowym zakresem ruchów, 1 gracz vs 1 gracz, wyświetlanie gry w konsoli znakami ASCII, sprawdzanie warunków ruchu. Sprawdzanie warunków wygranej. Naprzemienna gra graczy.

Kod najpierw sprawdza warunki końca gry (brak pionków albo brak możliwego ruchu), potem pobiera ruch od gracza, robi walidacje (czy pola są poprawne, czy start ma pionek gracza, czy cel jest pusty), sprawdza kierunek ruchu, wymusza bicie gdy jest możliwe, wykonuje zwykły ruch albo bicie (z możliwością kolejnych bic w petli), a na końcu zmienia gracza i odświeża planszę. Kod znajduje się w pliku warcaby.cpp, linie: 1367-1701

```
while(!koniec)
{
    char nie_gracz;
    if (gracz == 'O') nie_gracz = '@';
    else nie_gracz = 'O';

    if(bez_piona(war)) { koniec = true; break; }
    if(!czy_jest_ruch(war, gracz, nie_gracz)) { koniec = true; break; }

    string z, d;
    bool musi_bic = czy_musi_bic(war, gracz, nie_gracz);

    cout << "Ruch gracza: " << gracz << endl;

    cout << "Podaj pole startowe: "; cin >> z; czysc();
    cout << "Podaj pole docelowe: "; cin >> d; czysc();

    int zwier, zkol, dwier, dkol;
    bool OKz = pole_na_tab(z, zwier, zkol);
    bool OKd = pole_na_tab(d, dwier, dkol);
    if(!OKz || !OKd) { cout << "Niepoprawny foramt ruchu." << endl << endl;
continue; }

    int kier_weir = dwier - zwier;
    int kier_kol = dkol - zkol;
    int kier;
    if (gracz == 'O') kier = -1;
    else if (gracz == '@') kier = 1;

    if(kier_weir == kier && (kier_kol == 1 || kier_kol == -1))
    {
        if(musi_bic == false) { war[zwier][zkol] = '*'; war[dwier][dkol] =
gracz; }
        else { cout << "Masz bicie, musisz je wykonac" << endl << endl;
continue; }
```



```

    }
    else if(kier_weir == 2 * kier && (kier_kol == 2 || kier_kol == -2))
    {
        int wier_po_sr = (zwier + dwier) / 2;
        int kol_po_sr = (zkol + dkol) / 2;
        if(war[wier_po_sr][kol_po_sr] == nie_gracz) ruch_bicia(war, gracz,
nie_gracz, wier_po_sr, kol_po_sr, zwier, zkol, dwier, dkol);
        else { cout << "Nie ma tam przeciwnika do zbicia." << endl << endl;
continue; }
    }
    else { cout << "Niepoprawny ruch po skosie." << endl << endl; continue; }

    if(gracz == '0') gracz = '@';
    else gracz = '0';
}

```

3.1 SCREEN

```
C:\Windows\System32\cmd.e  X + v - □ X

      a b c d e f g h
+---+---+---+---+---+---+
1 | | @ | | @ | | @ | | @ | 1
+---+---+---+---+---+---+
2 | @ | | @ | | @ | | @ | 2
+---+---+---+---+---+---+
3 | | * | | * | | @ | | @ | 3
+---+---+---+---+---+---+
4 | 0 | | * | | * | | * | 4
+---+---+---+---+---+---+
5 | | @ | | * | | * | | * | 5
+---+---+---+---+---+---+
6 | * | | 0 | | * | | 0 | 6
+---+---+---+---+---+---+
7 | | 0 | | 0 | | 0 | | 0 | 7
+---+---+---+---+---+---+
8 | 0 | | 0 | | 0 | | 0 | 8
+---+---+---+---+---+---+
      a b c d e f g h

Ruch gracza: 0
Podaj pole startowe: a4
Podaj pole docelowe: b4
Pole docelowe jest niepoprawne, spróbuj ponownie.
Podaj pole startowe: |
```

```
C:\Windows\System32\cmd.e  X + v - □ X

      a b c d e f g h
+---+---+---+---+---+---+
1 | | @ | | @ | | @ | | @ | 1
+---+---+---+---+---+---+
2 | @ | | @ | | @ | | @ | 2
+---+---+---+---+---+---+
3 | | 0 | | * | | @ | | @ | 3
+---+---+---+---+---+---+
4 | * | | * | | * | | * | 4
+---+---+---+---+---+---+
5 | | @ | | * | | * | | * | 5
+---+---+---+---+---+---+
6 | * | | 0 | | * | | 0 | 6
+---+---+---+---+---+---+
7 | | 0 | | 0 | | 0 | | 0 | 7
+---+---+---+---+---+---+
8 | 0 | | 0 | | 0 | | 0 | 8
+---+---+---+---+---+---+
      a b c d e f g h

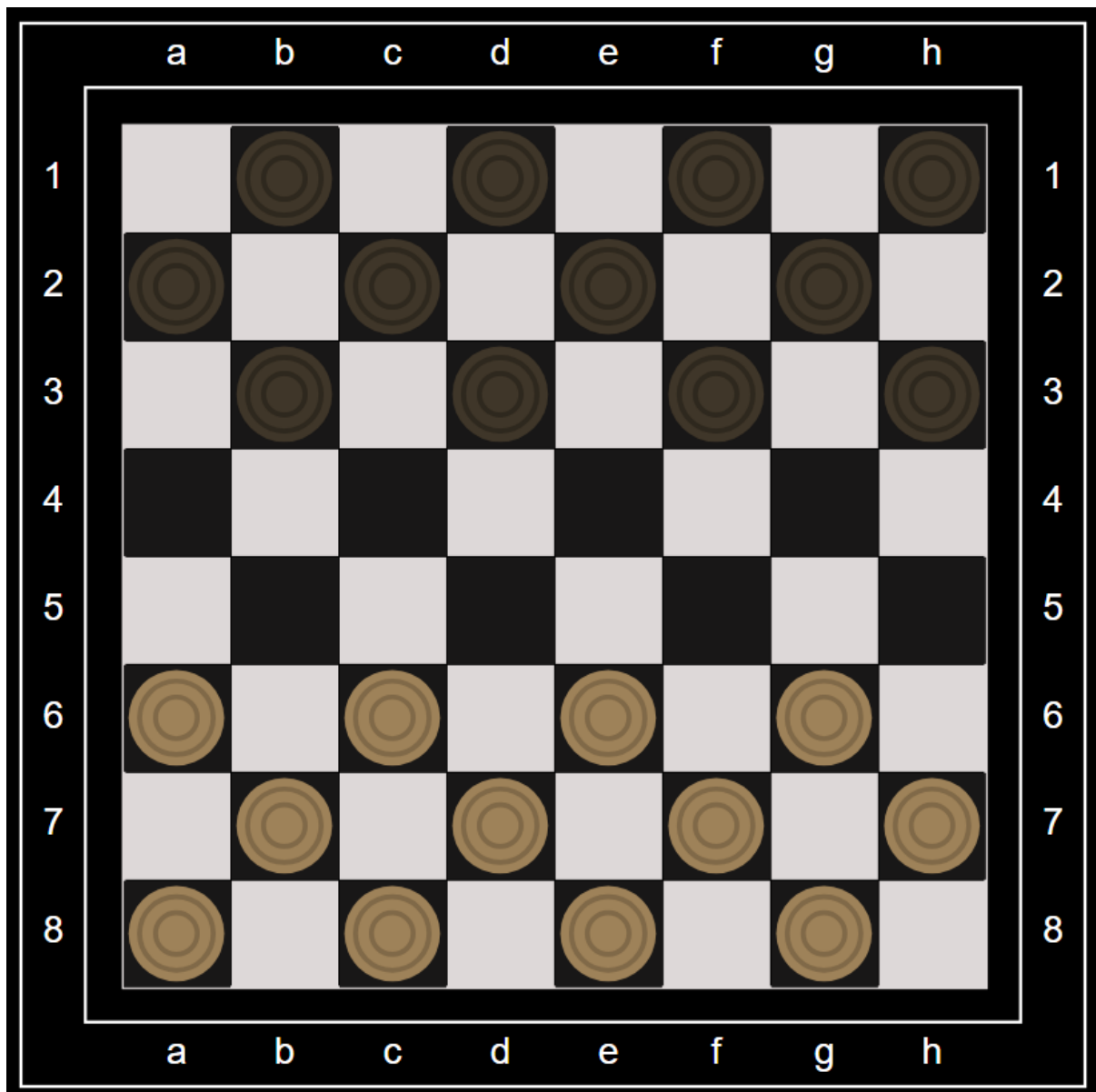
Ruch gracza: @
Podaj pole startowe: c2
Podaj pole docelowe: d3
Masz bicie, musisz je wykonać
Podaj pole startowe: |
```

3.2 Wyświetlanie planszy szachownicy w .SVG wraz z ustawieniem figur jako bardziej złożonych obrazków w odpowiednim polu.

Kod tworzy plik SVG z planszą do warcabow. Najpierw rysuje czarne tło i ramkę z kilku prostokątów. Potem ustawia rozmiar pola oraz tablice współrzędnych pól. Na końcu petla rysuje 64 pola, zmieniając kolor naprzemiennie, dzięki czemu powstaje szachownica. Na tak przygotowanych polach rysuje pionki na podstawie tablice charów.

```
plik << "<rect x=\"0\" y=\"0\" width=\"100%\" height=\"100%\" fill=\"#000000\"
/>\n";
plik << "<rect x=\"65\" y=\"65\" width=\"870\" height=\"870\" fill=\"#ffffff\"
/>\n";
plik << "<rect x=\"67.5\" y=\"67.5\" width=\"865\" height=\"865\"
fill=\"#000000\" />\n";
plik << "<rect x=\"117.5\" y=\"117.5\" width=\"765\" height=\"765\"
fill=\"#ffffff\" />\n";
plik << "<rect x=\"120\" y=\"120\" width=\"760\" height=\"760\"
fill=\"#000000\" />\n";
int R_pola = 87;
int polaX[8] = {150, 238, 326, 414, 502, 590, 678, 766};
int polaY[8] = {150, 238, 326, 414, 502, 590, 678, 766};
plik << "<rect x=\"150\" y=\"150\" width=\"703\" height=\"703\" fill=\"none\"
stroke=\"#ddd8d8ff\" stroke-width=\"3\" />\n";
for(int i = 0; i < 8; i++)
{
for(int j = 0; j < 8; j++)
{
string kolor;
if((i + j) % 2 == 0) { kolor = "#ddd8d8ff"; }
else { kolor = "#181717ff"; }
plik << "<rect x=\"" << polaX[j] << "\" y=\"" << polaY[i] << "\" width=\"" <<
R_pola << "\" height=\"" << R_pola << "\" fill=\"" << kolor << "\" />\n"; } }
```

3.2 SCREEN



3.3. KOD Obsługa zamiany pionka w „damkę” – inny typ pionka i inne warunki ruchów damki niż

pionka – „damka” porusza się tylko po przekątnej. Inna grafika „damki” niż pionka. Gdy przeciwnik zmieni swój pionek na damkę po przejściu na koniec planszy przeciwnik to jest w stanie poruszać się nie tylko do przodu, ale także do tyłu, jak i biec we wszystkie strony. Kod znajduje się w `warcaby.cpp`.

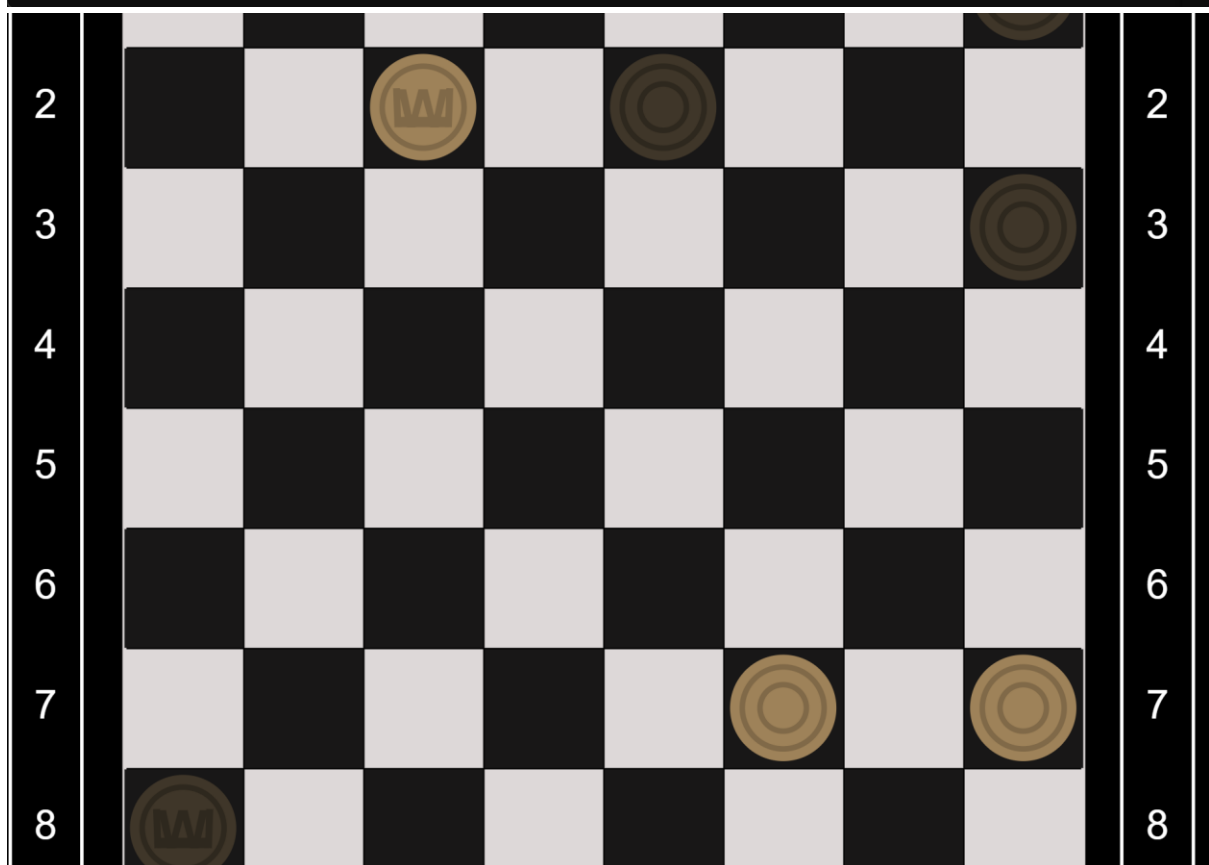
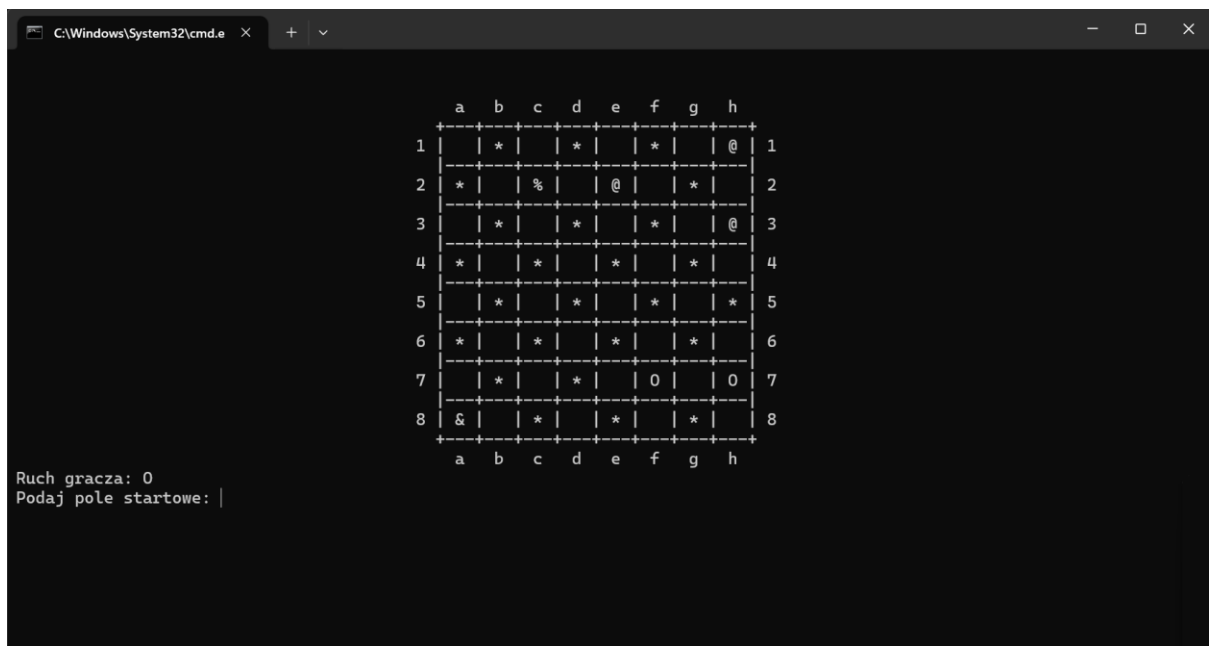
```
if(fig == '%')
{
    if(dwier - 2 >= 0 && dkol - 2 >= 0)
    {
```

```

        if( (war[dwier - 1][dkol - 1] == '@' || war[dwier - 1][dkol - 1]
== '&') &&
            (war[dwier - 2][dkol - 2] == '*') )
        {
            return true;
        }
    }
    if(dwier - 2 >= 0 && dkol + 2 <= 7)
    {
        if( (war[dwier - 1][dkol + 1] == '@' || war[dwier - 1][dkol + 1]
== '&') &&
            (war[dwier - 2][dkol + 2] == '*') )
        {
            return true;
        }
    }
    if(dwier + 2 <= 7 && dkol - 2 >= 0)
    {
        if( (war[dwier + 1][dkol - 1] == '@' || war[dwier + 1][dkol - 1]
== '&') &&
            (war[dwier + 2][dkol - 2] == '*') )
        {
            return true;
        }
    }
    if(dwier + 2 <= 7 && dkol + 2 <= 7)
    {
        if( (war[dwier + 1][dkol + 1] == '@' || war[dwier + 1][dkol + 1]
== '&') &&
            (war[dwier + 2][dkol + 2] == '*') )
        {
            return true;
        }
    }
    return false;
}

```

3.3 SCREEN



3.4KOD Zapis historii stanu gry do pliku i odczyt stanu gry z pliku i kontynuacja gry. Kod wczytuje do pliku aktualnego gracza, ustawienia i stan planszy a później a w dalszej czesci gra jest odpalana na podstawie tych informacji.

```

bool zapis_gry(const string& zapis, char war[8][8], char gracz, bool war_ods,
bool war_graf)
{
    ofstream plik(zapis.c_str());
    if(!plik.is_open())
    {
        return false;
    }

    plik << "gracz=" << gracz << "\n";

    int p_graf = 0;
    int p_ods = 0;
    if(war_graf) p_graf = 1;
    if(war_ods) p_ods = 1;

    plik << "graf=" << p_graf << "\n";
    plik << "ods=" << p_ods << "\n";

    for(int i = 0; i < 8; i++)
    {
        for(int j = 0; j < 8; j++)
        {
            char pust = war[i][j];
            if(pust == ' ') pust = '.';
            plik << pust;
        }
        plik << "\n";
    }

    return true;
}

```

3.4 SCREEN

```
C:\Windows\System32\cmd.e  X  +  v  -  □  X

\\\\\\\\\\\\ ZAPIS GRY \\\\\\\\\\\

- (1) SLOT 1 [zajety] -
- (2) SLOT 2 [pusty] -
- (3) SLOT 3 [pusty] -
- (4) SLOT 4 [pusty] -
- (P) Powrot do gry -
- (X) Powrot do MENU -

Wybierz opcje: |
```

```
C:\Windows\System32\cmd.e  X  +  v  -  □  X

\\\\\\\\\\\\ Wczytaj GRE \\\\\\\\\\\

- (1) SLOT 1 [zajety] -
- (2) SLOT 2 [pusty] -
- (3) SLOT 3 [pusty] -
- (4) SLOT 4 [pusty] -
- (X) Powrot do MENU -

Wybierz opcje: |
```